

Lightweight CNNs for Potato Leaf Disease Diagnosis on Commodity Hardware

From-scratch vs. transfer learning, with Grad-CAM interpretability

Niels Pacheco

MIA-07 — Redes Neuronales y Aprendizaje Profundo (Sección C)

Final Project · June 2026

Why potatoes? Why now?

- The **potato** (*Solanum tuberosum*) was domesticated in the **Peruvian Andes** ~7,000 years ago and is the world's **3rd most important food crop**.
- **Late blight** (*Phytophthora infestans*) caused the Irish famine and can **destroy a field in days**; **early blight** (*Alternaria solani*) is also widespread.
- Expert diagnosis is **scarce in rural Andean communities** — exactly where potato farming is concentrated.
- Deep learning could **democratise diagnosis from a single leaf photo** — *if* it runs on hardware farmers actually have.

The research question

How close can a small CNN trained *from scratch* on a GPU-less laptop come to ImageNet-pretrained transfer learning for potato leaf disease classification?

Two gaps in the literature we address:

1. Most systems assume **GPUs + large pretrained backbones**.
2. Headline accuracies rarely come with **confidence intervals, calibration, or a check that the model looks at the disease**.

Dataset: PlantVillage (potato subset)

- 2,152 images, 3 classes: **healthy** / **early blight** / **late blight**
- Lab-controlled photos, **class-imbalanced** (few healthy)
- **Fixed stratified split** 70/15/15 — *same test set for every model* (needed for valid significance testing)

PlantVillage potato leaf classes

Healthy



Early blight



Late blight



Four models, one fair comparison

Model	Source	Params
Custom CNN	from scratch	~0.33 M
MobileNetV2	ImageNet, frozen + new head	~2.2 M
ResNet-18	ImageNet, frozen + new head	~11 M
EfficientNet-B0	ImageNet, frozen + new head	~4 M

- Custom CNN: 4 conv blocks (Conv-BN-ReLU ×2 + pool) → global avg pool → dropout → linear
- Same preprocessing & ImageNet normalisation for all → **only architecture/weights differ**

Methodology — built for honesty & reproducibility

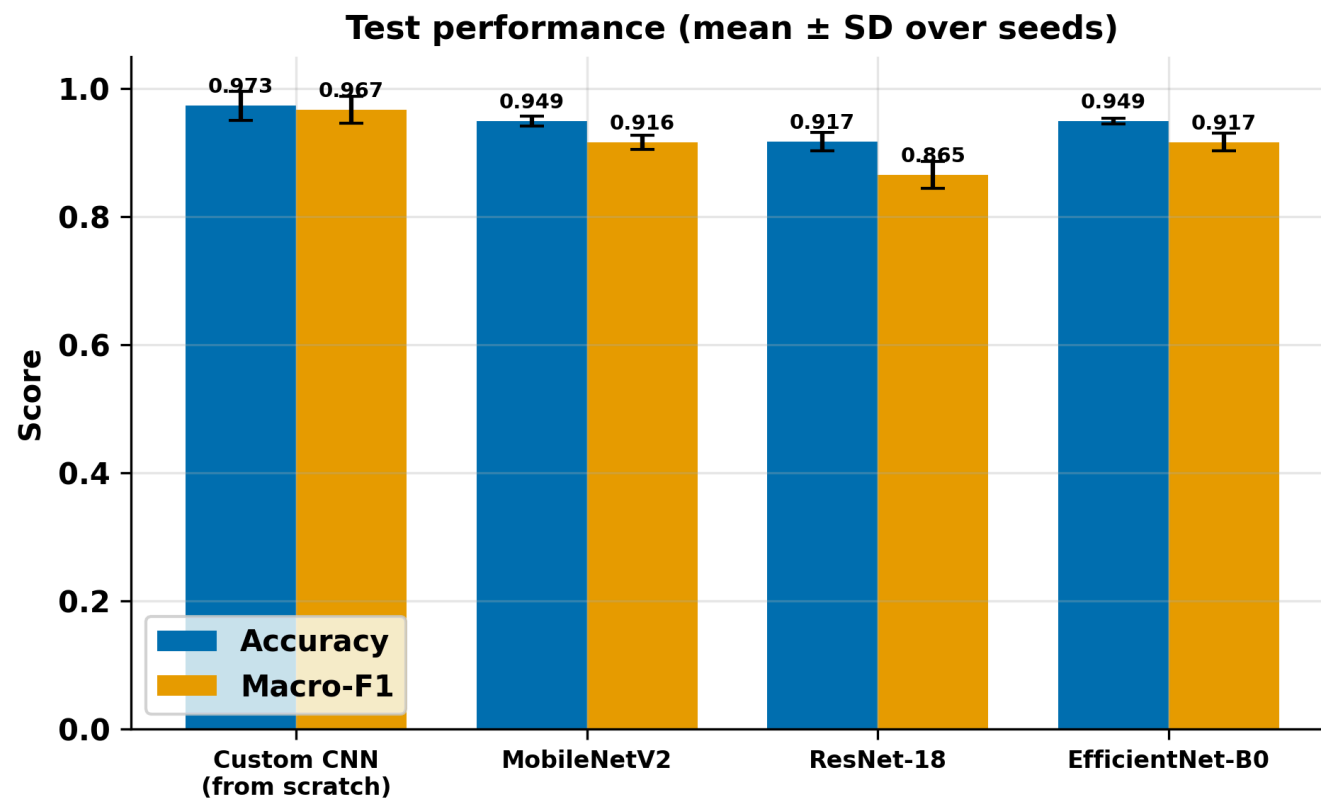
- **AdamW** + cosine schedule, **class-weighted** loss + label smoothing, early stopping on val macro-F1
- **3 random seeds** per model → report **mean ± SD** and **bootstrap 95% CIs**
- **McNemar's paired test** between models on the shared test set
- **Calibration**: Expected Calibration Error + reliability diagrams
- **Grad-CAM** to see *where the model looks*
- Trained entirely on **Apple-Silicon (MPS)**, no GPU

/autoresearch: autonomous hyperparameter search

- Seeded **random search**, 10 trials, selecting on **validation** macro-F1 (never the test set)
- Search space: learning rate, weight decay, width, dropout, augmentation strength
- Best config folded into `configs/custom_cnn.yaml`
- Full trajectory logged → `autoresearch.jsonl` + dashboard + worklog

Best validation macro-F1: **96.9%** (trial 9: lr 1e-3, wd 1e-5, width 24, dropout 0.3, aug 0.5)

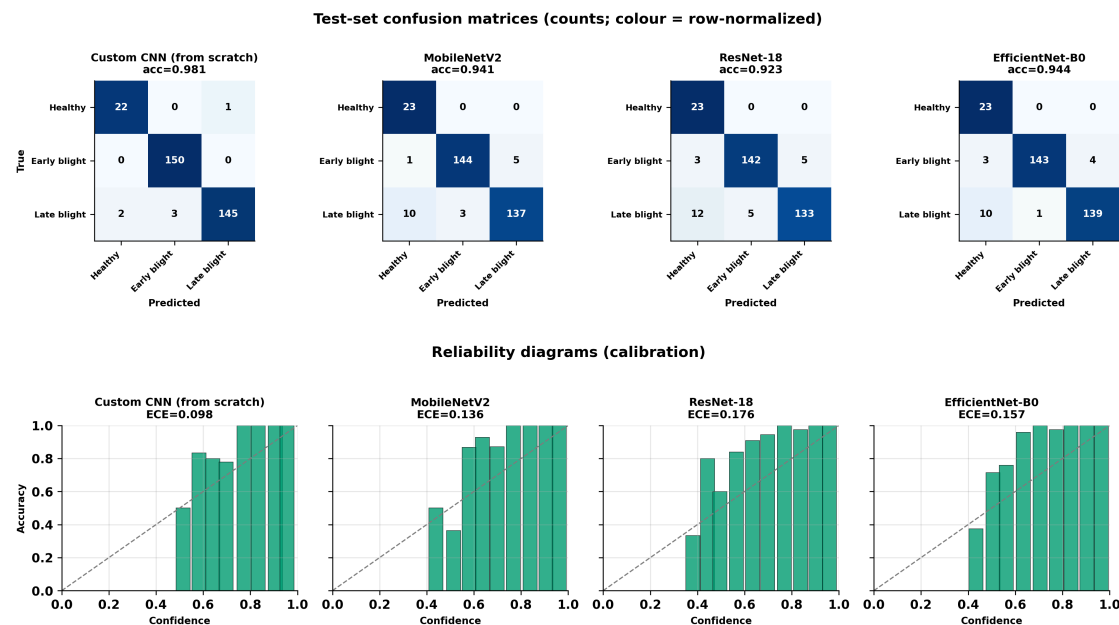
Results — the from-scratch CNN *wins* 🏆



- Custom CNN macro-F1 **96.7%** > best transfer **91.7%** (EfficientNet-B0), with **0.33M params** (7–34× smaller)

- **McNemar**: custom CNN significantly beats **all three** frozen transfer models ($p = 0.017, 0.004,$

Confusion structure & calibration



- Custom CNN errors are mostly **disease** ↔ **disease** (operationally benign — both need action)
- **!** Frozen transfer models misclassify some **late blight** → **healthy** (dangerous false negatives) — the custom CNN also *fails more safely*
- Models are **moderately miscalibrated** (ECE 0.12–0.17); custom CNN best-calibrated — post-hoc temperature scaling would help

🔍 The punchline: where do the models look?

Grad-CAM: where each model looks (per class)

Healthy



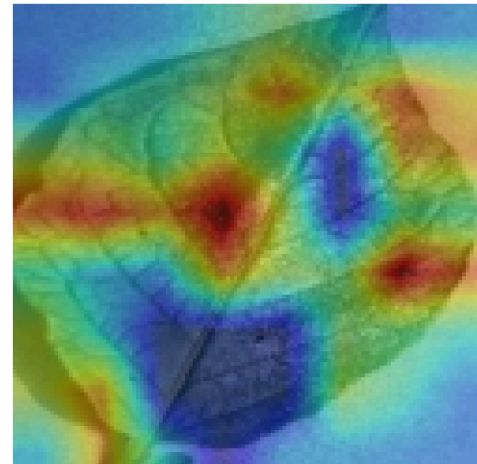
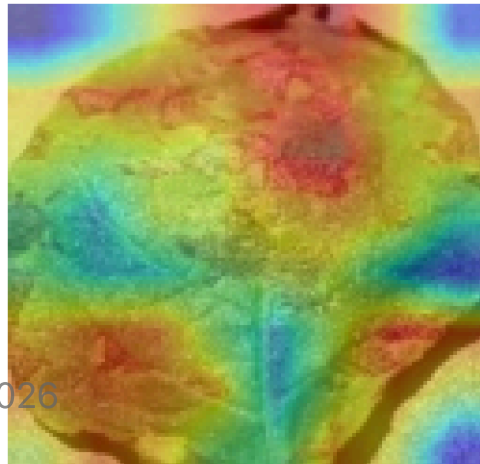
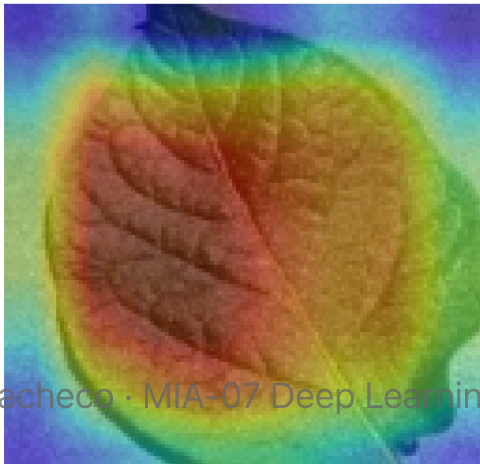
Early blight



Late blight



Custom CNN (from scratch)



Discussion

What we can claim

- A **small, scratch-trained CNN on a laptop** *significantly outperforms* frozen-backbone transfer learning here — at 7–34× fewer parameters.
- Under an **equal low-compute budget**, learning domain-specific features end-to-end beats reusing frozen ImageNet features.

What we must not claim

- That high PlantVillage accuracy $\Rightarrow$ field readiness. **Background bias** undermines that.
- That transfer learning is "worse" in general — **full fine-tuning** (more compute) would likely close/reverse the gap.

Limitations: lab images, easy 3-class task, frozen backbones, moderate calibration (ECE 0.12–0.17), no on-device deployment yet.

Conclusions & reproducibility

- Compact from-scratch CNN **significantly beats frozen transfer** here (equal low-compute budget); full fine-tuning would cost more.
- **Grad-CAM suggests a shared reliance on dataset background** — honesty over leaderboard chasing.
- Everything reproduces with **one command**: pinned `uv` env, fixed seeds, config-driven runs, auto-generated figures & paper.

```
make setup && make data && make train-all && make eval && make figures && make paper
```

Repo: github.com/nielspac177/papa-vision

Thank you — ¡Gracias!

Questions?

Niels Pacheco · nielspacheco1997@gmail.com

Future work: field images · background removal / segmentation · on-device latency & energy benchmarking